

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Topics To be Covered Today

- ☐ Lexical Issues
- ☐ Reserve Words
- ☐ Data Types



Tokens/ Lexical Issues

Tokens are the smallest meaningful symbols in the language.

Token type	Description/Purpose	Examples
Keywords	are the reserved names of a language and cannot be used as names of variables, functions etc. Words with special meaning to the compiler	int, double, for, while
Identifiers	refers to the names of variable, arrays, functions, classes, Interfaces etc.	std, x, myFunction, sum
Literals	refer to fixed values that we cannot change in a program.	"Hello, world!", 24.3, 0, 'c'
Operators	are special symbols which operate on variable & constants, and form an expression.	+, -, &&, %, <
Punctuation /Separators	Are the special characters used to separate statements. Punctuation defining the structure of a program	{ } () , ;
Whitespace	Spaces of various sorts; ignored by the compiler	Spaces, tabs, newlines, comments

Identifiers

An identifier is a word used in a program to name a variable, method (function), class, interface, package etc.

Identifiers

To name an identifier you must follow some rules:

- An identifier (a name) can contain only letters, digits 0 through 9, and the underscore character (_).
- The first character in an identifier cannot be a digit.
- In particular, no name can contain a space or any other special character other than \$ and underscore _.
- There is no limit to the length of an identifier.
- Java is case sensitive. That is, uppercase and lowercase letters are considered to be different characters. For example, Java considers mystuff, myStuff, and MyStuff to be three different identifiers, and you could have three different variables with these three names.
- Keywords cannot be used as identifiers.

Contin....

The following variable names are valid in Java:

idname_6

id_name

_idname

The following variable names are invalid in Java:

6_idname //Starting with a digit

idname# //Using a special character (#) such as %, *, #, and so on

id name //Using space

Reserve Words

- Reserved words or keywords are words that have a specific meaning to the compiler and cannot be used for other purposes in the program.

- For example:

When the compiler sees the word `class`, it understands that the word after `class` is the name for the class.

Reserve Words

- There are 50 reserved keywords in Java.
- The keyword `const` and `goto` are reserved but not used.
- All keywords are in lower case.
- `true`, `false`, and `null` are literals not keywords .
- **We can't use these reserved names as Identifiers (Variable or Class or Interface names).**

Keywords

abstract	<u>boolean</u>	break	byte	case	catch	char
class	const	continue	default	do	double	else
extends	final	finally	float	for	goto	if
implements	import	instanceof	int	interface	long	native
new	package	private	protected	public	return	short
static	super	switch	synchronized	this	throw	throws
transient	try	void	volatile	while		

Modifiers

- Java uses certain reserved words called modifiers that specify the properties of the data, methods, and classes and how they can be used.
- Examples of modifiers are public and static. Other modifiers are private, final, abstract, and protected. A **public** method, or class can be accessed by other programs/ classes. A **private** method cannot be accessed by other programs. The code is only accessible within the declared class.

Data Types

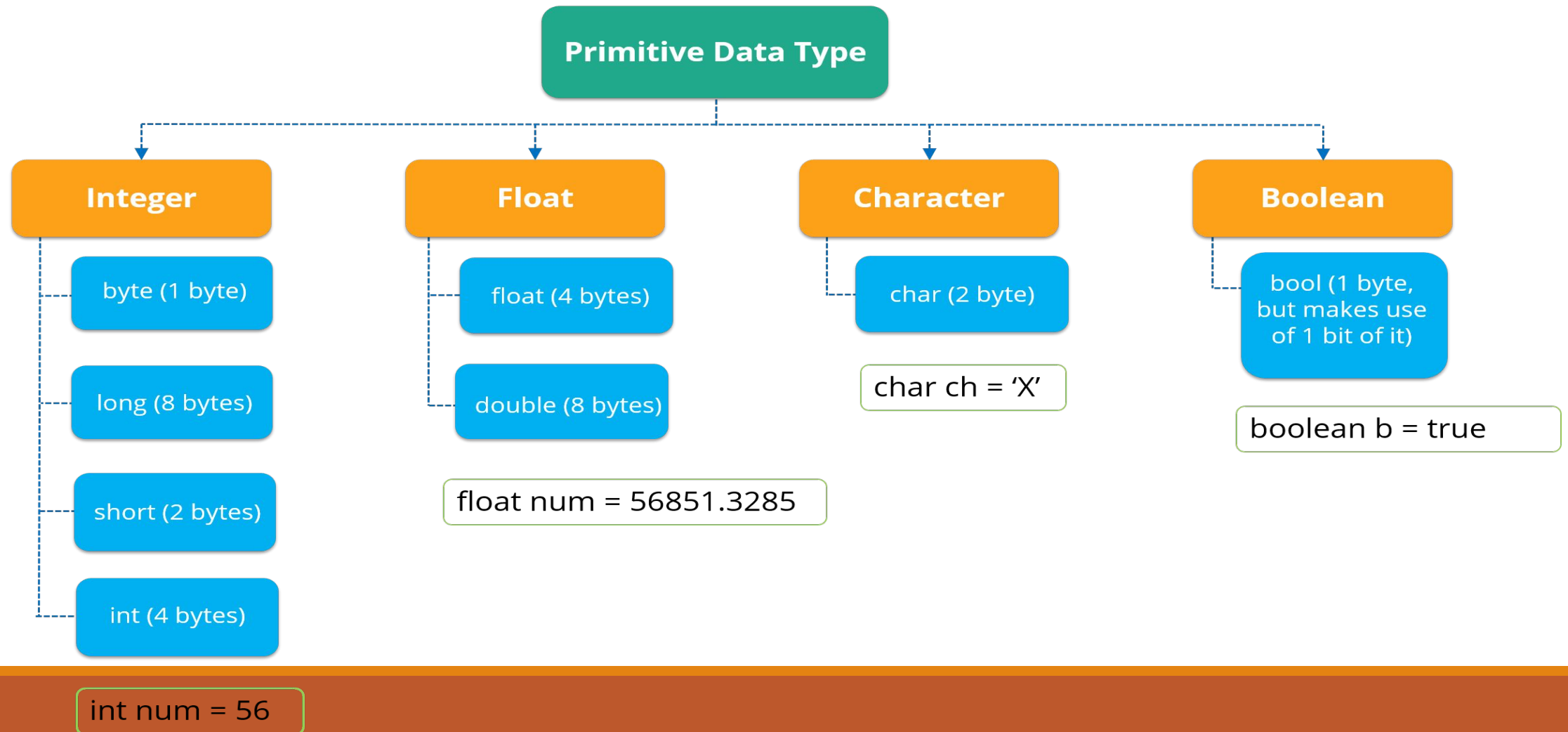
Data type describes the **size** and **type** of values that can be stored in a variable.

In any program, you need to store particular type of data in the computer's memory. The compiler should know the amount of memory that has to be allocated to that particular data. It directs the compiler to allocate specific amount of memory to a particular type of data.

Data types in Java can be broadly classified in two categories:

- **Primitive data types/ Simple data types**
- **Non-Primitive Data Types/ Derived Data Types or Reference**

Primitive Data Types: for storing simple values and are predefined in java



Boolean Data Type

A boolean data type comprises of a bit of information and can store only **true** or **false** values. This data type is used to track **true/false conditions**. Now let's write a small program and understand how it works.

```
1  class booleanDataType{
2  public static void main(String args[]){
3  // Setting the values for boolean data type
4
5  boolean Java = true;
6  boolean Python = false;
7  System.out.println(Java);    // Output will be true
8  System.out.println(Python);  // Output will be false
9  }
10 }
```

Byte Data Type

This is an example of a primitive data type. It is an 8-bit signed two's complement integer. It stores whole numbers that lie between -128 to 127. A byte data type is helpful for saving memory in large amounts. Now let's write a small program and understand how it works.

```
1  class ByteExample {  
2  public static void main(String[] args) {  
3  byte n, a;  
4  n = 127;  
5  a=177;  
6  System.out.println(n); // prints 127  
7  System.out.println(a); // throws an error because it cannot store more than 127  
8  }  
9  }
```

Char

This data type is used to store a **single** character. The character must be enclosed within single quotes, like 'E' or 'e'. Alternatively, you can also use ASCII values to display certain characters. Let's take a small example and see how it works.

```
char  alpha = 'J';  
char  a = 65, b = 66, c = 67;  
System.out.println(alpha); // prints J  
  
System.out.println(a); // Displays A  
System.out.println(b); // Displays B  
System.out.println(c); // Displays C
```

Short data type

A short data type is greater than byte in terms of size and less than an integer. It stores the value that ranges from -32,768 to 32,767. The default size of this data type: 2 bytes. Let's take an example and understand the short data type.

```
1  class ShortExample {  
2  public static void main(String[] args) {  
3  short n= 3435,  
4  System.out.println(n); // prints the value present in n i.e. 3435  
5  }  
6  }
```


Int Data type

This data type can store whole numbers from -2147483648 to 2147483647. Generally, int is the preferred data type when you create **variables** with a numeric value.

For example:

```
1 | int num = 5464564;  
2 | System.out.println(num); // prints 5464564
```

long data type

This data type is a 64-bit two's complement integer. By default, the size of a long data type is 64 bit and its value ranges from -2^{63} to $2^{63}-1$.

For example:

```
1 | long num = 150000000000L;  
2 | System.out.println(num); // prints 150000000000
```

Floating Data types

You should use a floating point type whenever you need a number with a decimal, such as 8.88 or 3.14515.

float data type

This data type can store fractional numbers from $3.4e-038$ to $3.4e+038$. Note that you should end the value with an "f". Let's take a small example and understand this data type in a detailed manner.

```
1 | float num =67;  
2 | System.out.println(num); // prints the floating number value
```

Double Data type

The double data type can store fractional numbers from $1.7\text{e}-308$ to $1.7\text{e}+308$. Note that you should end the value with a “d”:

```
1 | double num = 79.678d;  
2 | System.out.println(num); // prints double value
```

Non-Primitive Data Types:

Used for storing complex objects. Created by the programmer and is not defined by Java.

They are also called “reference variables” or “object references” since they reference a memory location which stores the data.

